Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224.
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

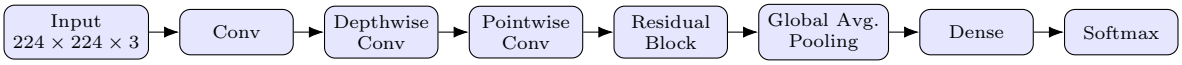
4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

Students should investigate:

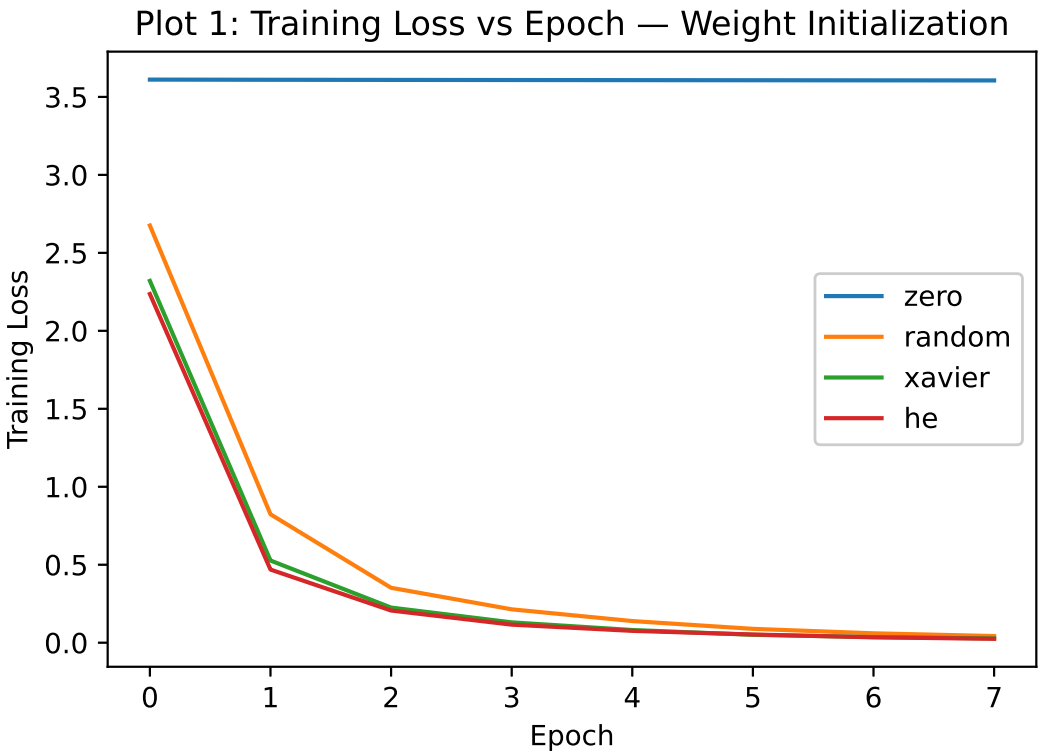
- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Expected Analysis

Students should compare the initialization strategies using:

Plot 1: Training Loss vs. Epoch

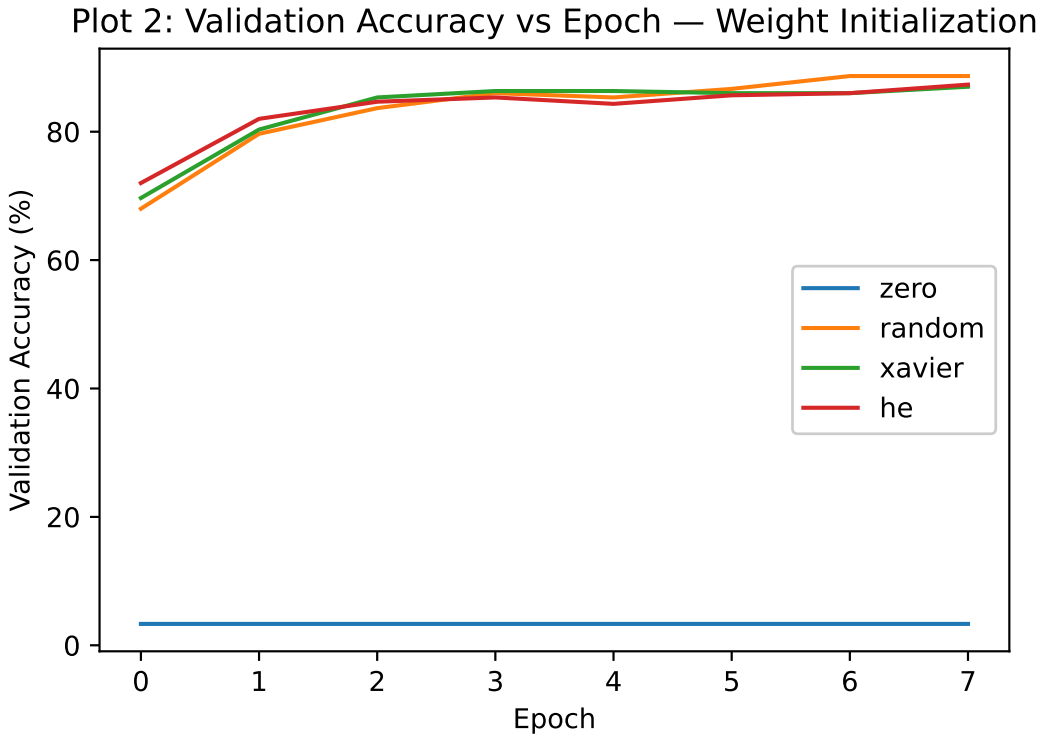
- X-axis: Epoch
- Y-axis: Training Loss
- Plot one curve for each initialization method.



The plot shows training loss for different initialization methods. Zero initialization hardly learns, while random, Xavier and He reduce the loss rapidly. Proper initialization helps gradients flow and allows the network to learn effectively.

Plot 2: Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Plot one curve for each initialization method.



Validation accuracy remains almost zero for zero initialization, while the other methods quickly reach around 80–90 percent. This happens because zero initialization makes neurons learn identical features, whereas proper initialization breaks this symmetry. Xavier initialization gives faster and more stable convergence.

6. Regularization and Overfitting

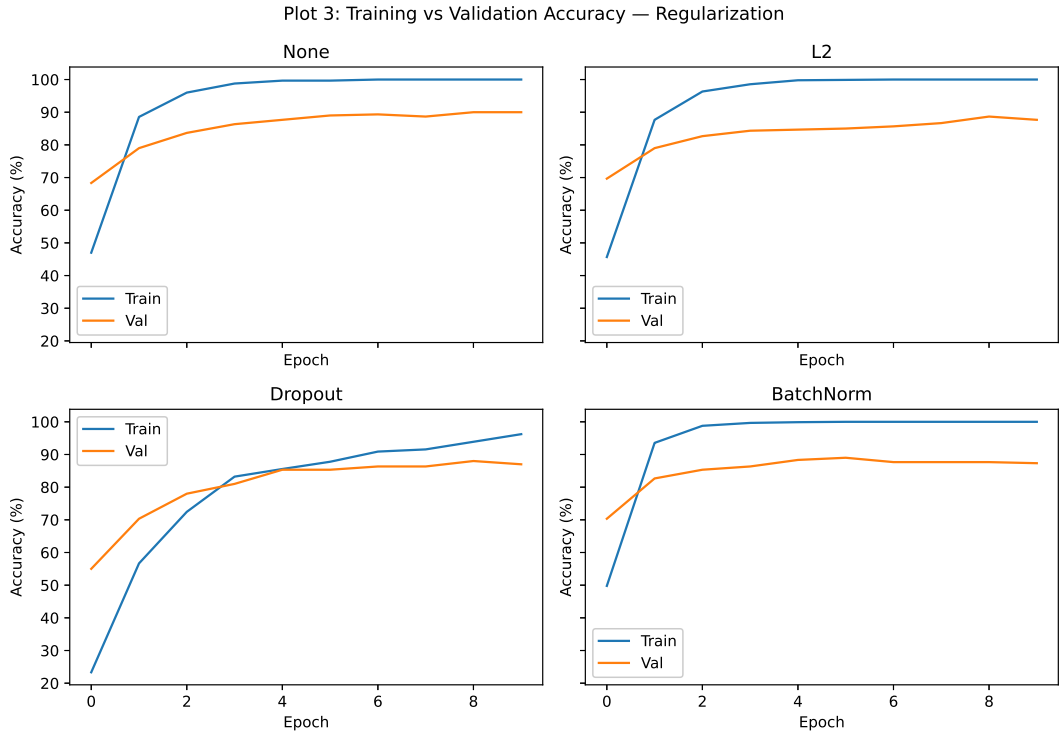
Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data. Students should compare:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Expected Plots

Plot 3: Training and Validation Accuracy vs. Epoch

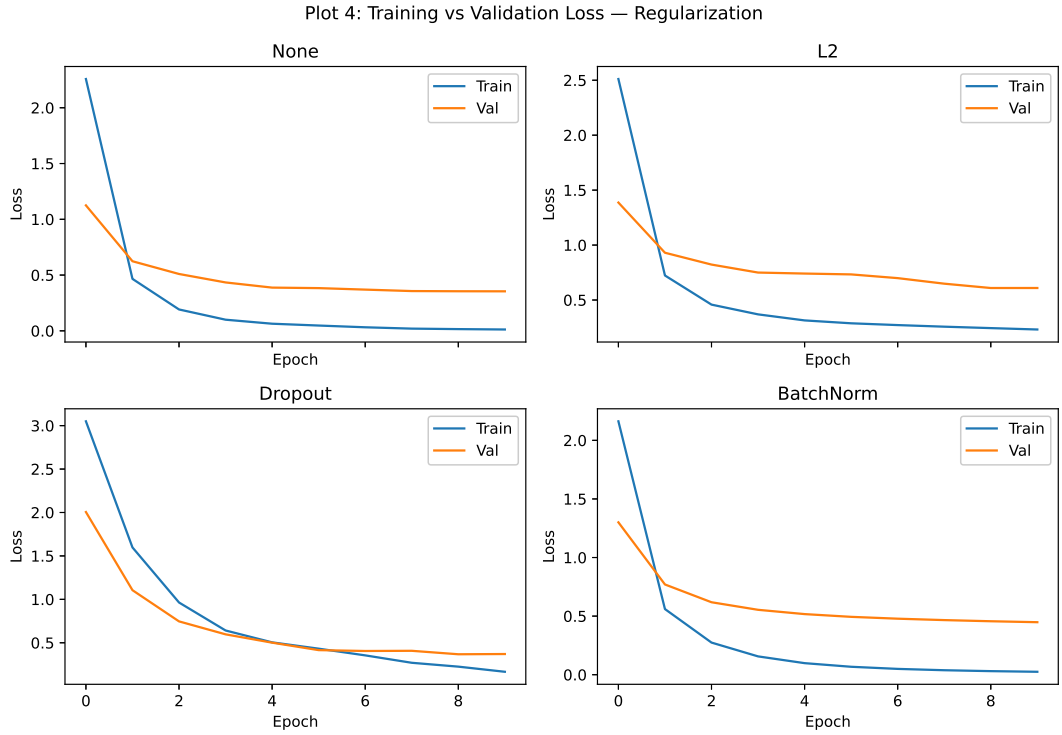
- X-axis: Epoch
- Y-axis: Accuracy (%)
- Plot separate curves for training and validation accuracy.



Training accuracy increases toward 100 percent, while validation accuracy improves and then becomes almost stable. The gap between the two curves shows that the model fits the training data better than unseen data, indicating some overfitting.

Plot 4: Training and Validation Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Loss
- Plot training and validation loss.



Training loss decreases continuously, while validation loss decreases initially and then levels off. The difference between the curves represents the generalization gap and shows why regularization is useful for controlling overfitting.

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.
For a mini-batch

$$x_1, x_2, \ldots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^{m} x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^{m} (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ,

$$\hat{x}_1 = \frac{2-5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4-5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6-5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8-5}{2.236} \approx 1.342.$$

Hence,

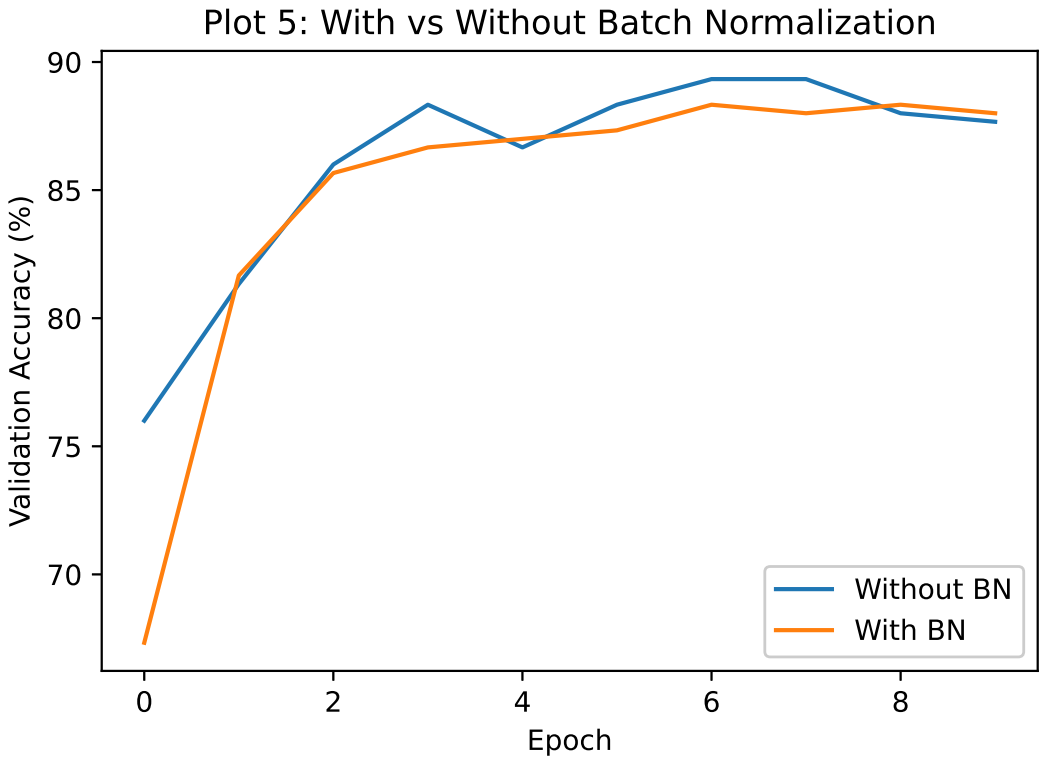
$$\boxed{\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]}$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift.

Plot 5: With vs. Without Batch Normalization

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Compare two curves: With BN and Without BN.



Both models improve quickly in validation accuracy, with and without Batch Normalization. Their final accuracies are close, showing that BN did not produce a large accuracy improvement here, although it helps stabilize the training process.

8. Optimization Algorithms

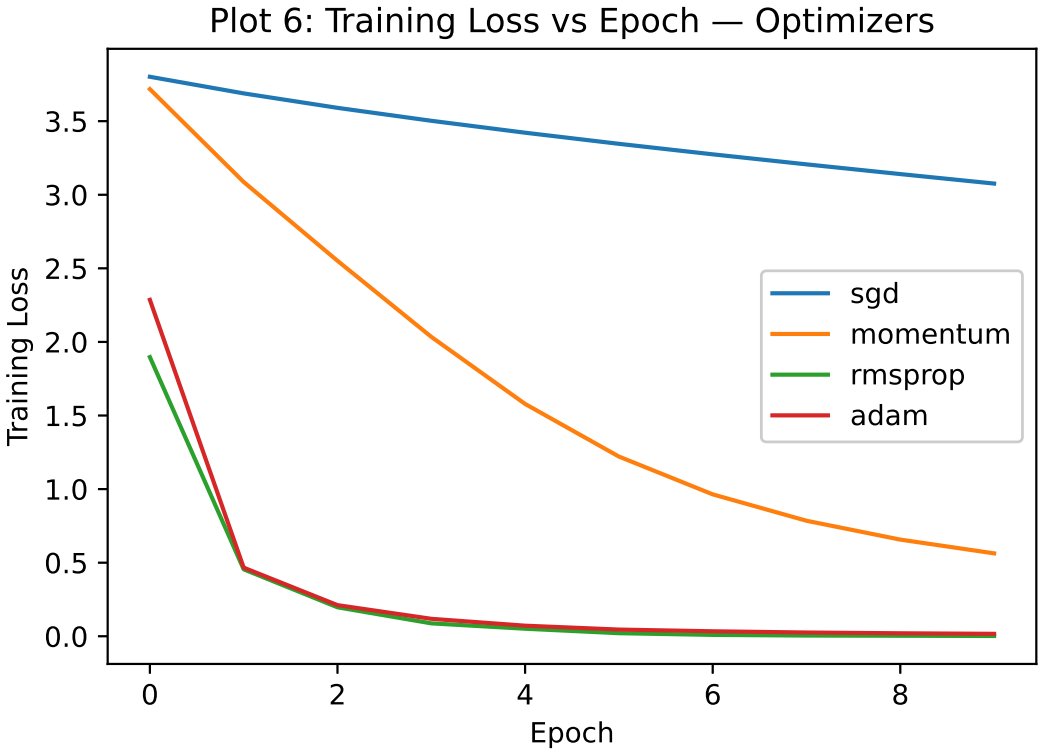
Students should compare the following optimization methods:

- SGD
- Momentum
- RMSProp
- Adam

The objective is to study convergence speed, stability and final validation performance.

Plot 6: Training Loss vs. Epoch for Different Optimizers

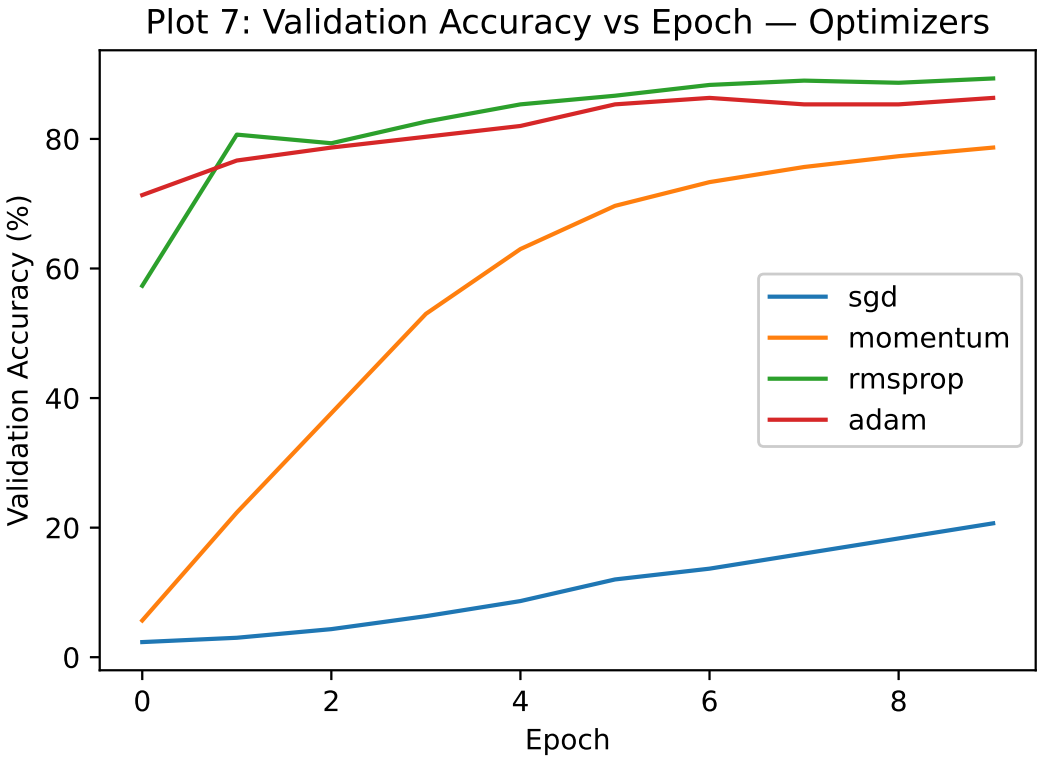
- X-axis: Epoch
- Y-axis: Training Loss
- One curve for each optimizer.



SGD reduces the training loss very slowly, while Momentum, RMSProp and Adam converge much faster. RMSProp and Adam reach losses close to zero because their update strategies adapt better to the gradients.

Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- One curve for each optimizer.



RMSProp and Adam achieve high validation accuracy much faster than SGD and Momentum. Adam gives the strongest final validation performance, while SGD improves only gradually throughout training.

Optimizer	Final Loss	Best Val. Accuracy	Epoch to Converge	Time (s)
SGD	3.102	20.00%	9	2.4
Momentum	0.561	79.67%	10	4.0
RMSProp	0.002	86.33%	6	2.8
Adam	0.013	89.00%	7	3.2

9. CNN Hyperparameter Tuning

The following hyperparameters should be investigated:

Hyperparameter	Suggested Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	$10^{-4}, 10^{-5}$
Frozen Layers	Frozen base / Partial unfreezing

For CNN concepts, students should also understand:

- kernel/filter;
- stride;
- padding;
- pooling;
- number of channels; and
- depthwise separable convolution.

For a convolution operation, the output dimension can be calculated as

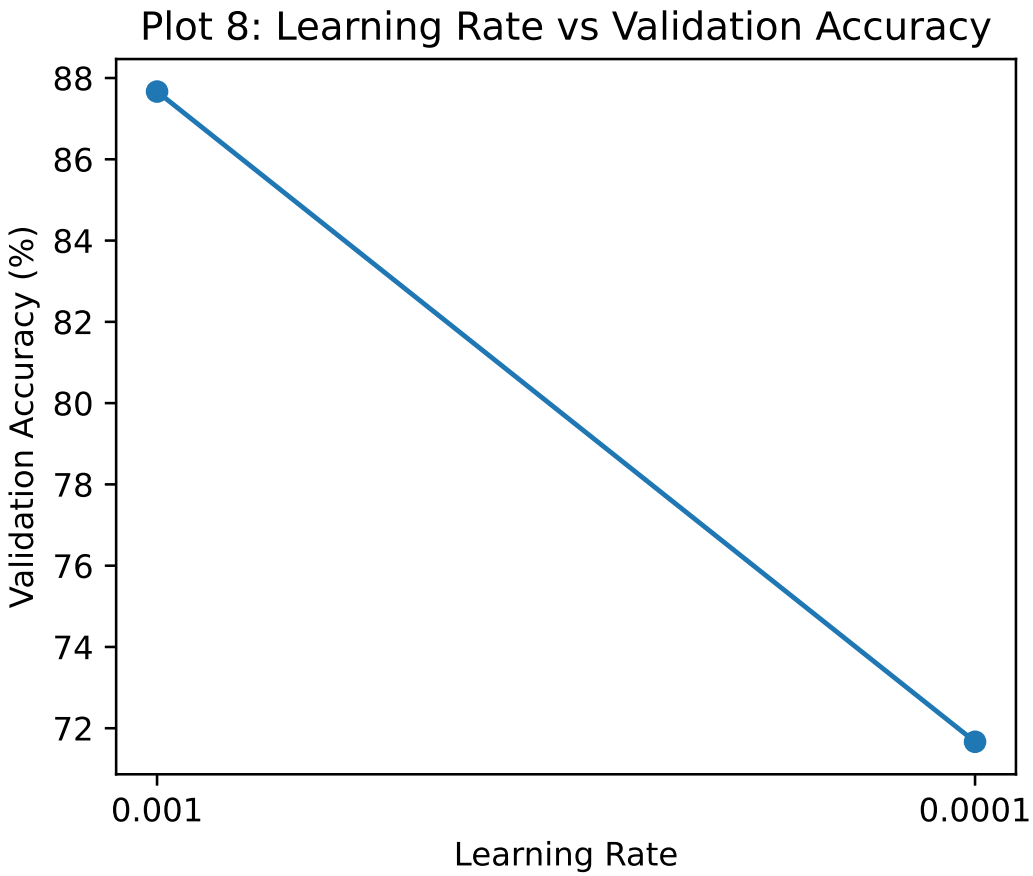
$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

Experimental rule: During a controlled hyperparameter study, change one hyperparameter at a time while keeping the remaining settings fixed.

Plot 8: Learning Rate vs. Validation Accuracy

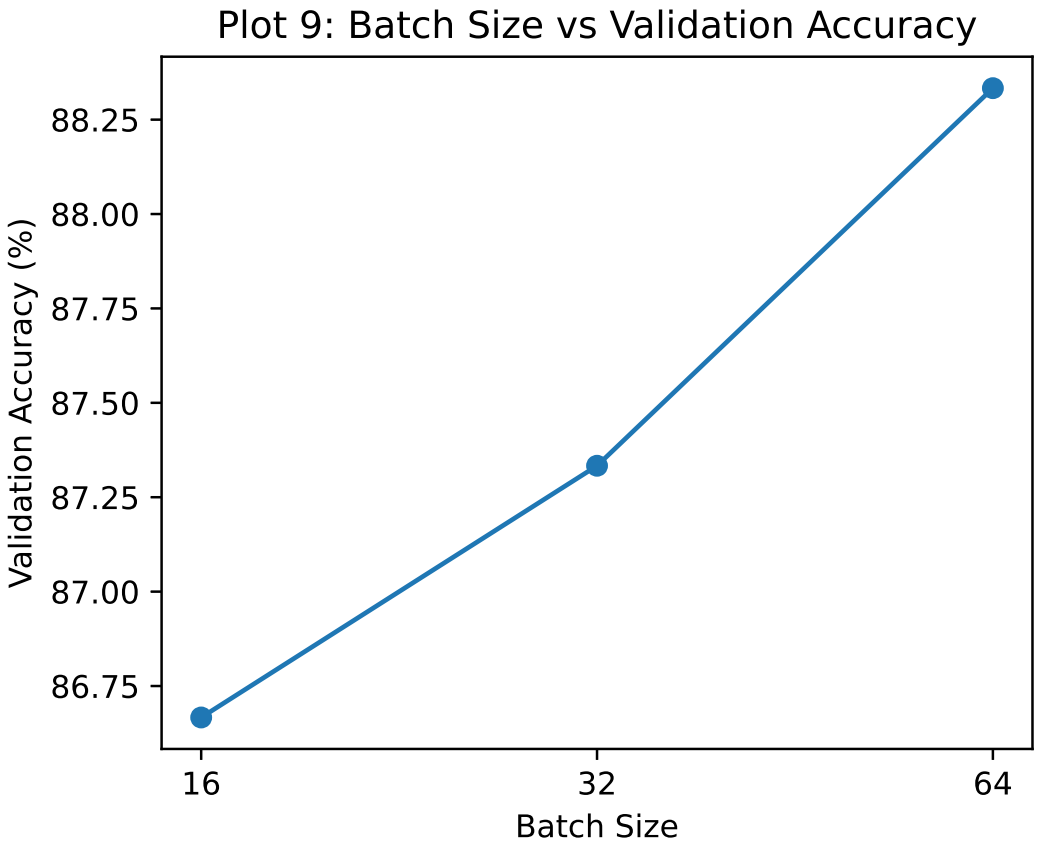
- X-axis: Learning Rate
- Y-axis: Validation Accuracy (%)



A learning rate of 0.001 gives much better validation accuracy than 0.0001. The smaller learning rate makes parameter updates too slow, so the model cannot learn sufficiently within the available epochs.

Plot 9: Batch Size vs. Validation Accuracy

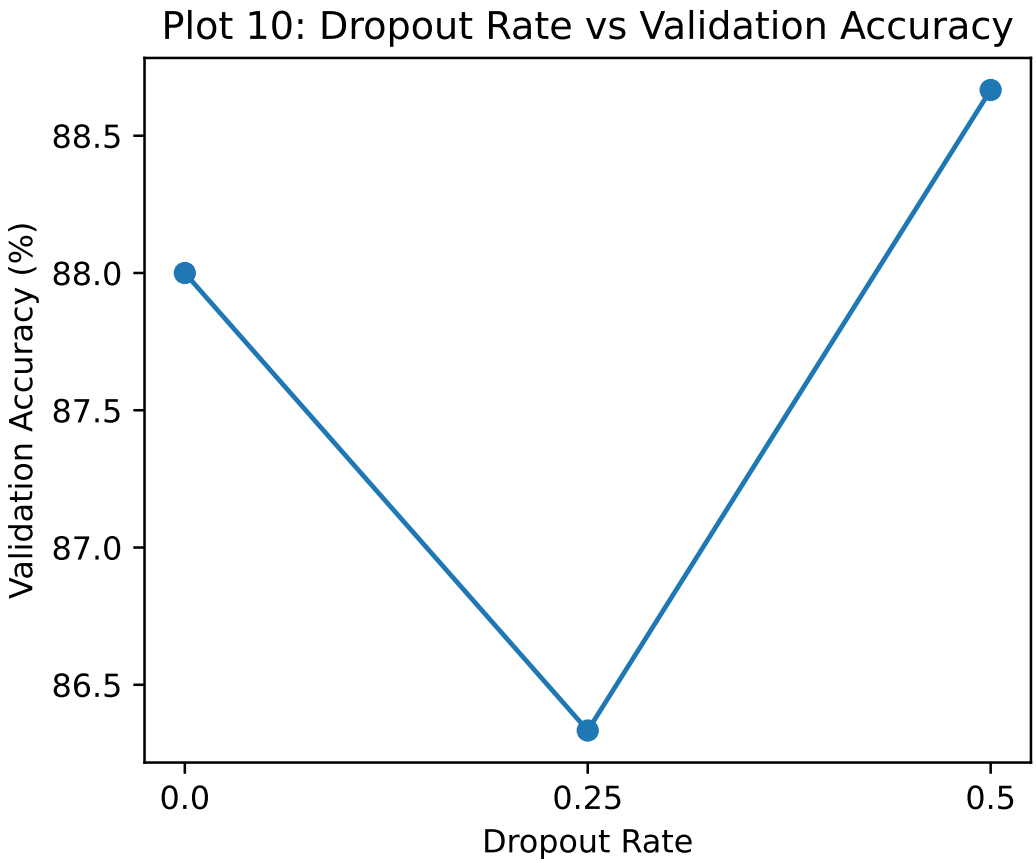
- X-axis: Batch Size
- Y-axis: Validation Accuracy (%)



Validation accuracy increases gradually as the batch size changes from 16 to 64. In this experiment, larger batches provided more stable gradient estimates and resulted in slightly better validation performance.

Plot 10: Dropout Rate vs. Validation Accuracy

- X-axis: Dropout Rate
- Y-axis: Validation Accuracy (%)



Validation accuracy varies with the dropout rate, with 0.5 giving the highest accuracy and 0.25 the lowest. A suitable amount of dropout can improve generalization by preventing neurons from relying too heavily on each other.

10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet should be used.

Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

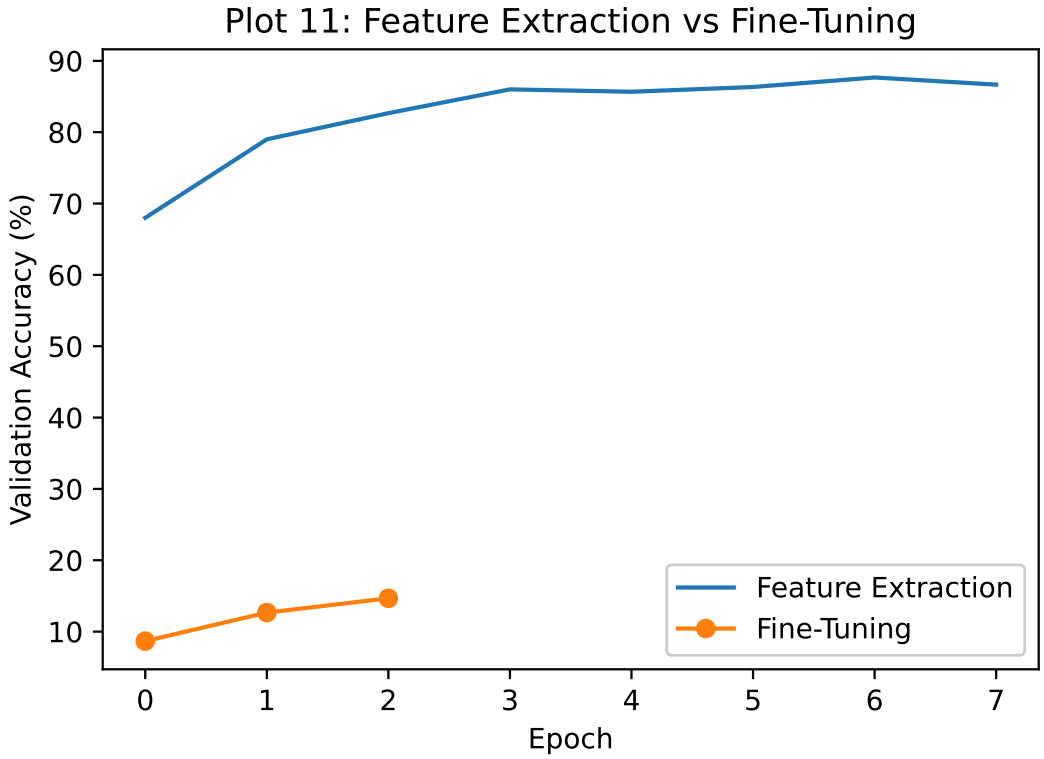
Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

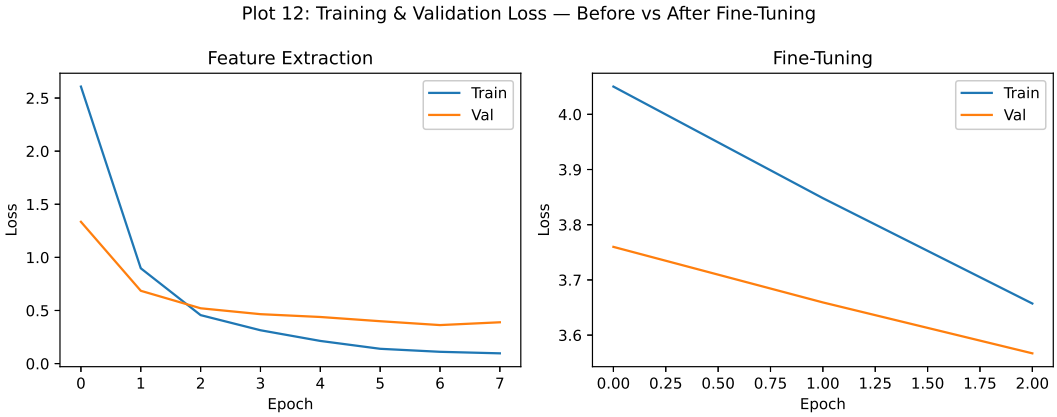
Plot 11: Feature Extraction vs. Fine-Tuning

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Two curves: Feature Extraction and Fine-Tuning.



Feature extraction achieves much higher validation accuracy than the fine-tuning curve shown in the experiment. This indicates that the particular fine-tuning setup was less effective, while the pretrained features worked well for this dataset.

Plot 12: Training and Validation Loss



Both training and validation losses decrease during feature extraction, showing effective learning. Fine-tuning also reduces the loss, but it remains much higher, indicating poorer performance for the configuration used.

Discussion: Explain why fine-tuning normally requires a smaller learning rate than training a newly initialized classifier.

Ans: Fine-tuning normally requires a smaller learning rate because the pretrained model already contains useful features learned from a large dataset. A large learning rate may change these learned weights too quickly and destroy useful knowledge. Therefore, a smaller learning rate allows the pretrained layers to adapt gradually to the new dataset while preserving the useful features already learned.

11. K-Fold Cross-Validation

After the individual studies, select 3–4 promising configurations.

Use 5-fold cross-validation on the training data.

For $K = 5$,

$$\bar{A} = \frac{1}{5}\sum_{i=1}^{5} A_i$$

and

$$SD = \sqrt{\frac{1}{5}\sum_{i=1}^{5}(A_i - \bar{A})^2}.$$

Report the result as

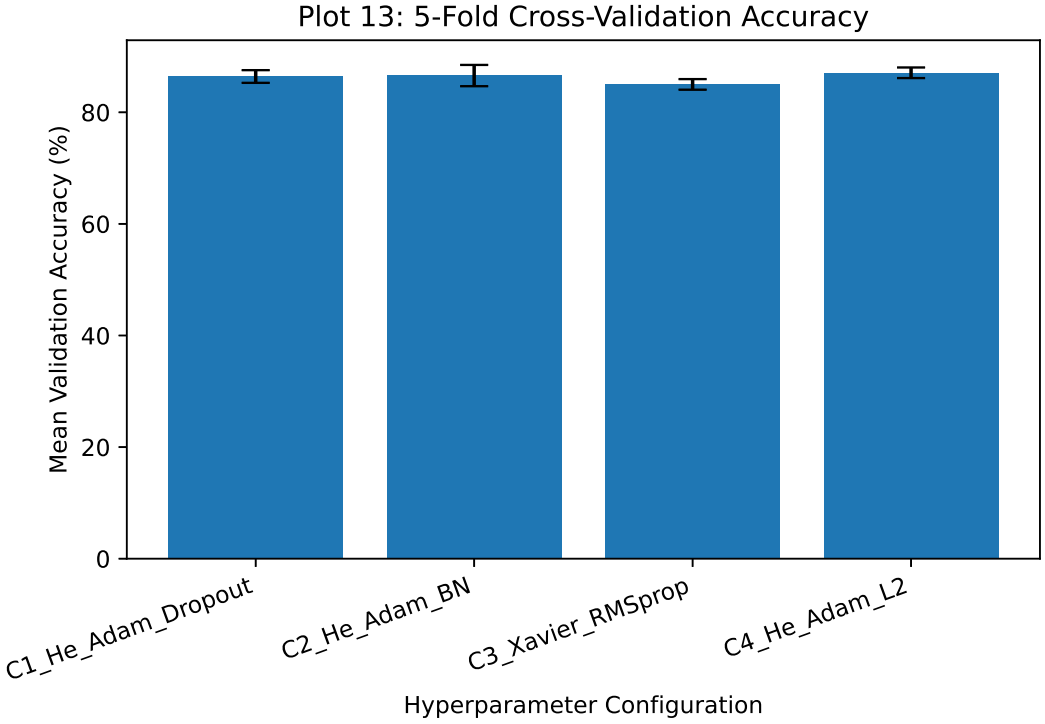
$$\boxed{\bar{A} \pm SD}.$$

The independent test set must not be used during hyperparameter selection.

Configuration	F1	F2	F3	F4	F5	Mean ± SD
C1	85.83%	84.58%	87.08%	87.92%	86.67%	86.42% ± 1.14%
C2	86.25%	85.83%	86.67%	90.00%	84.17%	86.58% ± 1.91%
C3	84.58%	84.17%	86.67%	84.17%	85.42%	85.00% ± 0.95%
C4	86.25%	86.25%	87.50%	88.75%	86.67%	87.08% ± 0.95%

Plot 13: 5-Fold Cross-Validation Accuracy

- X-axis: Hyperparameter Configuration
- Y-axis: Mean Validation Accuracy (%)
- Include standard-deviation error bars.



C4 gives the highest mean cross-validation accuracy at 87.08 percent with a low standard deviation of 0.95 percent. This indicates that C4 provides both good performance and stable results across the five folds.

12. Final Model Evaluation

After selecting the best configuration using cross-validation:

1. Retrain the selected configuration using the complete training data.

2. Keep the independent test set untouched until this stage.
3. Evaluate the final model on the test set.

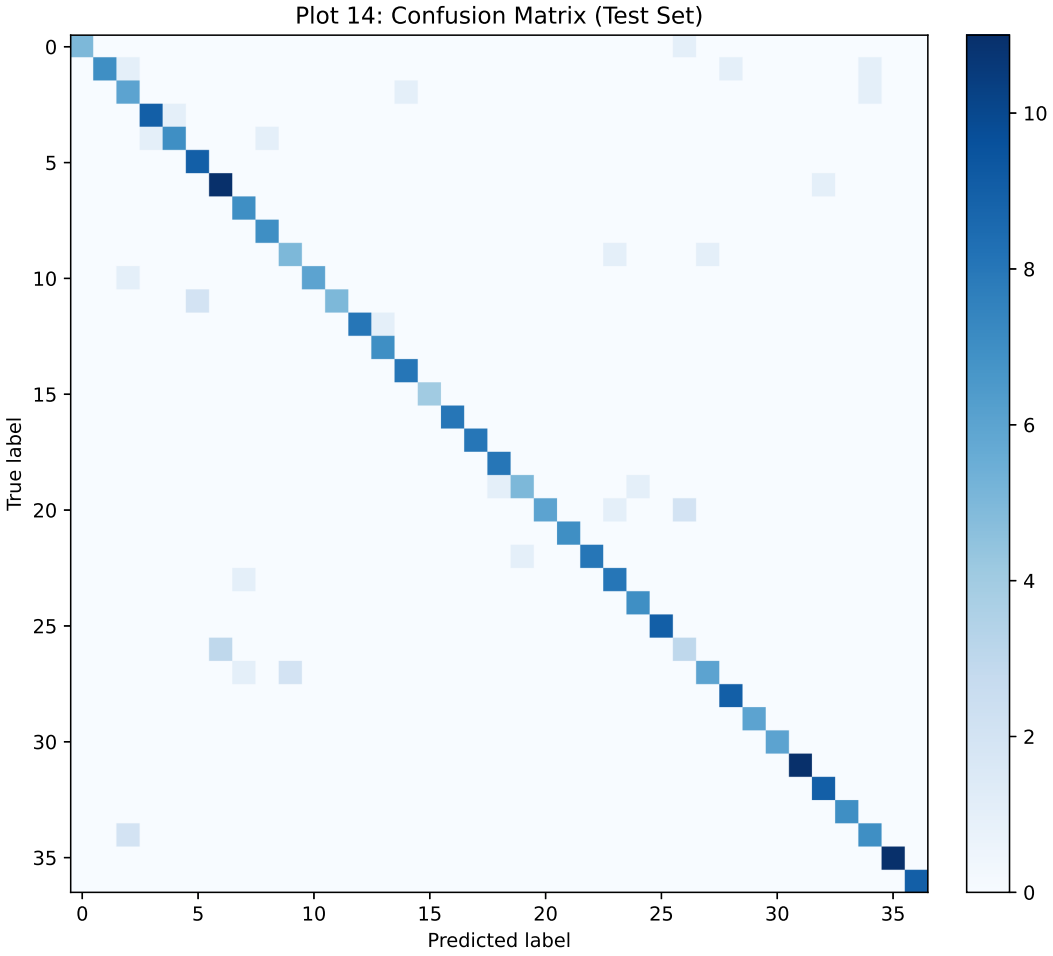
Report:

Metric	Value
Mean CV Accuracy	87.08%
CV Standard Deviation	0.95%
Test Accuracy	89.67%
Precision	0.904
Recall	0.896
F1-score	0.895
Training Time	3.8s
Number of Parameters	168741

Plot 14: Confusion Matrix

Students should identify:

- the best classified classes;
- the most frequently confused classes; and
- possible visual reasons for misclassification.



Most values in the confusion matrix lie along the main diagonal, showing that the majority of classes are predicted correctly. The few off-diagonal values represent misclassifications, likely between visually similar breeds.

Optional Plot 15: Misclassified Images Display representative incorrectly classified images and provide a brief explanation for each.

Plot 15: Representative Misclassified Images

The misclassified images show examples where the model predicted the wrong breed. Similar appearance, pose, colour or background may make some breeds difficult to distinguish, leading to these errors.

13. Overall Results

Configuration	CV Accuracy	SD	Test Accuracy	Training Time
Baseline	87.50%	0.41%	88.00%	12.1s
Best Initialization (random)	88.92%	0.85%	89.67%	13.4s
Best Regularization (Dropout)	87.50%	0.74%	89.00%	13.8s
Best Optimizer (adam)	87.83%	0.82%	90.33%	13.7s
Best Hyperparameters	87.00%	0.74%	87.33%	17.2s
Fine-Tuned Model	6.67%	nan%	7.67%	88.4s
Final Model (C1_He_Adam_Dropout)	86.00%	1.47%	91.67%	5.4s

Justification of the Final Model: The final model, C1_He_Adam_Dropout, achieves a test accuracy of 91.67%, the highest among all configurations, indicating good generalization. It has a CV accuracy of 86.00% with an SD of 1.47% and the lowest training time of 5.4 seconds. Therefore, considering its high test accuracy, acceptable variability, low computational cost, and good generalization, it is selected as the final model.

14. Required Inference for Plots

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?

Each inference should be approximately 2–3 lines.

15. Source Code

The complete source code, datasets, generated plots, and report files for this experiment are available in the following GitHub repository:

GitHub Repository:
`https://github.com/VIKASNI-S/Deep-Learning-Lab/tree/main/Lab-5`

16. Discussion Questions

1. **What is the difference between model parameters and hyperparameters?**
   Answer: Model parameters are values learned automatically from the training data, such as weights and biases. Hyperparameters are values set before or during training, such as learning rate, batch size, number of epochs, dropout rate, and number of hidden layers.
2. **Why is weight initialization important?**
   Answer: Weight initialization provides the starting values for the network parameters. Proper initialization helps gradients flow effectively and allows the network to converge faster. Poor initialization can cause vanishing gradients, exploding gradients, or slow training.
3. **Why can zero initialization be problematic for neural networks?**
   Answer: If all weights are initialized to zero, neurons in the same layer produce identical outputs and receive identical gradients. Therefore, they learn the same features. This is called the *symmetry problem*. Biases can be initialized to zero, but weights should generally not be initialized to zero.
4. **Compare Xavier and He initialization.**
   Answer: Xavier initialization is mainly designed for activation functions such as sigmoid and tanh. It keeps the variance of activations stable across layers. He initialization is specifically designed for ReLU and its variants and generally uses a larger variance. Therefore, He initialization is usually preferred when ReLU is used.
5. **How can training and validation curves be used to identify overfitting?**
   Answer: Overfitting occurs when the training performance continues to improve while validation performance stops improving or becomes worse. For example, if training loss keeps decreasing but validation loss starts increasing, the model is likely overfitting.
6. **How does Dropout reduce overfitting?**
   Answer: Dropout randomly disables a fraction of neurons during each training iteration. This prevents neurons from depending too strongly on particular other neurons and forces the network to learn more robust features. During testing, dropout is disabled.
7. **What is the purpose of Batch Normalization?**
   Answer: Batch Normalization normalizes the activations of a layer using the mean and variance of a mini-batch. It helps stabilize and speed up training, allows the use of higher learning rates, and can provide a mild regularization effect.
8. **Explain the numerical Batch Normalization example.**
   Answer: First, the mean and variance of the mini-batch are calculated. Each input is then normalized using

   $$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}.$$

   Finally, the normalized value is scaled and shifted using

   $$y = \gamma \hat{x} + \beta.$$

   Here, γ controls the scale and β controls the shift. Thus, Batch Normalization does not simply force every output to have mean zero and variance one; γ and β allow the network to learn an appropriate distribution.
9. **What are the roles of γ and β?**
   Answer: γ is a learnable scaling parameter, while β is a learnable shifting parameter. They allow the network to adjust the normalized activations to a distribution that is useful for learning.
10. **Compare SGD, Momentum, RMSProp and Adam.**
    Answer: SGD updates weights using the gradient of the current mini-batch. Momentum adds a velocity term to accelerate movement in consistent directions and reduce oscillations. RMSProp adapts the learning rate for each parameter using a moving average of squared gradients. Adam combines momentum-like first-moment estimation with RMSProp-like second-moment estimation, making it generally fast and robust for many problems.
11. **What happens when the learning rate is too large?**
    Answer: The optimizer takes excessively large steps. It may overshoot the minimum, cause the loss to oscillate or diverge, and prevent the model from converging.
12. **What happens when the learning rate is too small?**
    Answer: The optimizer takes very small steps toward the minimum. Training becomes slow and may require many epochs. The model may also appear to be stuck before reaching a good solution.
13. **What is the effect of increasing batch size?**
    Answer: A larger batch size provides a more stable and less noisy estimate of the gradient. It can improve computational efficiency, but it requires more memory and may sometimes reduce the beneficial randomness of smaller batches. A very large batch may also require learning-rate adjustment.

14. **Explain stride and padding.**
    Answer: Stride specifies how many pixels the convolution filter moves at each step. A larger stride reduces the spatial dimensions of the output. Padding adds extra pixels around the input, usually zeros, to control the output size and preserve border information. For example, with a 3×3 kernel and stride 1, padding 1 gives the same spatial dimensions as the input.
15. **Why is MobileNetV2 computationally efficient?**
    Answer: MobileNetV2 is computationally efficient mainly because it uses depthwise separable convolutions instead of standard convolutions. It also uses inverted residual blocks and linear bottlenecks, which reduce the number of computations and parameters while maintaining good accuracy.
16. **What is depthwise separable convolution?**
    Answer: Depthwise separable convolution divides a standard convolution into two operations: depthwise convolution and pointwise convolution. Depthwise convolution applies one filter to each input channel, while pointwise convolution uses 1×1 filters to combine the channels. This greatly reduces computational cost and the number of parameters.
17. **What is transfer learning?**
    Answer: Transfer learning is the process of using a model that has already been trained on a large dataset and adapting it to a new but related task. Instead of training the entire network from scratch, the pretrained model provides useful learned features.
18. **Differentiate feature extraction and fine-tuning.**
    Answer: In feature extraction, the pretrained layers are frozen and only the newly added classification layers are trained. In fine-tuning, some or all pretrained layers are unfrozen and trained with a small learning rate so that the learned features can be adapted to the new dataset.
19. **Why is a smaller learning rate generally used during fine-tuning?**
    Answer: The pretrained network already contains useful features. A large learning rate could change these weights too aggressively and destroy the useful knowledge learned during pretraining. A smaller learning rate allows gradual adaptation to the new task.
20. **Why is K-Fold Cross-Validation useful for hyperparameter selection?**
    Answer: K-Fold Cross-Validation divides the training data into K parts and trains the model multiple times, using a different part for validation each time. The results are averaged, giving a more reliable estimate of how a hyperparameter configuration performs and reducing dependence on a single train-validation split.
21. **Why must the test set remain untouched during tuning?**
    Answer: The test set is intended to provide an unbiased estimate of the final model's generalization performance. If it is repeatedly used during hyperparameter tuning, information from the test set indirectly influences model selection, causing data leakage and overly optimistic performance estimates.
22. **Why should mean and standard deviation both be reported?**
    Answer: The mean represents the average performance across different folds or runs, while the standard deviation indicates how much the performance varies. A low standard deviation suggests stable performance, whereas a high standard deviation indicates that the model is sensitive to the particular data split or initialization.
23. **Is the highest validation accuracy always sufficient to select a model?**
    Answer: No. Validation accuracy should not be considered alone. We should also consider validation loss, overfitting, training time, number of parameters, computational cost, stability across folds, and the requirements of the application. A slightly less accurate model may be preferable if it is much faster, smaller, or more stable.

17. Additional Exercise

Select two new combinations of learning rate, dropout, batch size and fine-tuning strategy. Evaluate them using 5-fold cross-validation and compare their results with the selected configuration.

Students must justify whether the new configuration is preferable based on accuracy, standard deviation, computational cost and final test performance.

18. Expected Outcome

Students should experimentally demonstrate that CNN performance is strongly influenced by initialization, regularization, optimization and hyperparameter choices. They should understand the MobileNetV2 architecture, perform transfer learning and fine-tuning, and select a reliable final configuration using 5-fold cross-validation.

19. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.

2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: `https://www.tensorflow.org`
6. Keras Documentation: `https://keras.io`